

Guía de sentencias CRUD para una base de datos en Mongo DB

Usaremos la base de datos: **Empresa**

Colecciones: **Empleado, Puesto, Salario y Categoria**

Primeras sentencias en la base de datos

Para crear una base de datos, use el comando "use" (usar). Si la base de datos no existe, el clúster de MongoDB la creará.

use Empresa

Para crear las colecciones utilizaremos la siguiente sentencia:

```
db.createCollection("Empleado");
```

```
db.createCollection("Puesto");
```

```
db.createCollection("Salario");
```

```
db.createCollection("Categoria");
```

Para observar las colecciones existentes en una base de datos se utiliza el comando.

```
show collections;
```

Muestra todas las funciones que podemos hacer a la base de datos en uso.

```
db.help()
```

Muestra todas las funciones que le podemos hacer a una colección.

```
db.getCollectionNames()
```

Muestra las estadísticas de la base de datos.

```
db.stats();
```

Permite borrar la base de datos que se encuentre por defecto, en caso que no se haya indicado ninguna base de datos borraría "test".

```
db.dropDatabase()
```

Permite crear una colección en una base de datos.

```
db.createCollection(Name,Options)
```

Sentencias CRUD

Insertar documentos

Colección Departamento

```
db.Departamento.insertMany([
  { _id:1, puesto: "Tecnologia"},
  { _id:2, puesto: "Ventas"},
  { _id:3, puesto: "Finanzas"}]);
```

Colección Categoria

```
db.Categoria.insertMany([
  { _id:1, desc_categoria: "oficinal"},
  { _id:2, desc_categoria: "operativo"},
  { _id:3, desc_categoria: "supervisor"},
  { _id:4, desc_categoria: "jefe"},
  { _id:5, desc_categoria: "director"},
  { _id:6, desc_categoria: "gerente"}]);
```

Colección Puesto

```
db.Puesto.insertMany([
  { _id:1, desc_puesto: "oficinista sucursal",salario:450000.00,Categoria:{_id:1},Departamento:{_id:1}},
  { _id:2, desc_puesto: "operativo sucursal",salario:350000.00,Categoria:{_id:2},Departamento:{_id:1}},
  { _id:3, desc_puesto: "supervisor sucursal",salario:650000.00,Categoria:{_id:3},Departamento:{_id:1}},
  { _id:4, desc_puesto: "jefe sucursal",salario:750000.00,Categoria:{_id:4},Departamento:{_id:1}},
  { _id:5, desc_puesto: "director sucursal",salario:850000.00,Categoria:{_id:5},Departamento:{_id:1}},
  { _id:6, desc_puesto: "gerente general",salario:950000.00,Categoria:{_id:6},Departamento:{_id:1}}]);
```

Colección Empleado

```
db.Empleado.insertMany([
  { _id:1, nombres:{nombre:"Carlos", apellido1:"Rojas", apellido2:"Coto"}, puesto:{_id:1}},
  { _id:2, nombres:{nombre:"Ana", apellido1:"Salas", apellido2:"Rojas"}, puesto:{_id:2}},
  { _id:3, nombres:{nombre:"Rosa", apellido1:"Mora", apellido2:"Ramos"}, puesto:{_id:3}},
  { _id:4, nombres:{nombre:"Juan", apellido1:"Torres", apellido2:"Vega"}, puesto:{_id:3}},
  { _id:5, nombres:{nombre:"Luis", apellido1:"Bonilla", apellido2:"Hidalgo"}, puesto:{_id:2}},
  { _id:5, nombres:{nombre:"Rudy", apellido1:"Barrantes", apellido2:"Rivas"}, puesto:{_id:4}},
  { _id:6, nombres:{nombre:"Roberto", apellido1:"Morales", apellido2:"Brenes"}, puesto:{_id:5}}]);
```

Insertar un solo registro en la colección Empleado

```
db.Empleado.insertOne({ _id:6, nombres:{nombre:"Andrea", apellido1:"Rojas", apellido2:"Mata"}, puesto:{_id:1}});
db.Empleado.insertOne({ _id:7, nombres:{nombre:"Rodrigo", apellido1:"Rojas", apellido2:"Coto"}, puesto:{_id:1}});
db.Departamento.insertOne({_id:4, puesto:"Recurso Humanos"});
```

Modificar documentos

Permite modificar el contenido de una colección.

```
db.Empleado.updateOne({_id:8},{ $set:{ "nombres.apellido2":"Mata" }});
```

Borrar documentos

Permite borrar un documento en particular

```
db.Departamento.deleteOne({_id:4});
```

Consultar documentos

Permite mostrar el contenido de un solo documento en particular.

```
db.Empleado.findOne({"_id":1});
```

Muestra el contenido de un empleado en particular:

```
db.Empleado.find({_id:1});
```

```
db.Empleado.find({_id:2});
```

```
db.getCollection("Empleado").find({_id:1});
```

Muestre el contenido de los empleados que tengan un nombre en particular:

```
db.Empleado.find({"nombres.nombre":"Carlos"});
```

```
db.getCollection("Empleado").find({"nombres.nombre":"Carlos"});
```

Muestre el contenido de los empleados que tengan un apellido en particular:

```
db.Empleado.find({"nombres.apellido1":"Rojas"});
```

La siguiente operación utiliza el `$in` operador para devolver documentos en la [colección de Empleado](#) donde `_id` es igual a `5`:

```
db.Empleado.find({_id:{$in:[5]}});
```

La siguiente operación utiliza el `$gt` operador devuelve todos los documentos de la colección de Empleado donde id mayor que `5`:

```
db.Empleado.find({_id:{$gt:5}});
```

El método `distinct()` se utiliza para obtener registros únicos.

```
db.Empleado.distinct("puesto")
```

Este método clasifica los documentos de salida en orden ascendente (1) o descendente(-1).

```
db.Puesto.find({}).sort({desc_puesto:1})
```

La siguiente operación utiliza el `$regex` operador para devolver documentos en la colección de Empleado donde `nombres.apellido2` comienza con la letra C(o es "LIKE C%")

```
db.Empleado.find({"nombres.apellido2":{$regex:/^C/}});
```

La siguiente operación utiliza el `$regex` operador para devolver documentos en la colección de Empleado donde `nombres.apellido1` comienza con la letra R(o es "LIKE R%")

```
db.Empleado.find({"nombres.apellido1":{$regex:/^R/}});
```

Combine operadores de comparación para especificar rangos para un campo. La siguiente operación regresa de los documentos de [colección de Puesto](#) donde `Salario` entre 350,000.00 y 1000000.00 (exclusivo):

```
db.Puesto.find( {salario:{ $gt: 350000.00, $lt:1000000.00}});
```

Consultas combinadas entre colecciones

La siguiente consulta utiliza el método `aggregate` para mostrar los documentos de **Empleado** que se relacionan con la colección **Puesto**, mediante el campo `puesto._id`, esto por cuanto el campo se había incluido originalmente en una lista de {}.

```
db.Empleado.aggregate([
  { $lookup:
    {
      from: "Puesto",
      localField: "puesto._id",
      foreignField: "_id",
      as: "comments"
    }
  }
]);
```

La siguiente consulta utiliza el método `aggregate` para mostrar los documentos de **Puesto** que se relacionan con la colección **Categoria**, mediante el campo **Categoria._id**, esto por cuanto el campo se había incluido originalmente en una lista de {}.

```
db.Puesto.aggregate([
  { $lookup:
    {
      from: 'Categoria',
      localField: 'Categoria._id',
      foreignField: '_id',
      as: 'comments'
    }
  }
]).pretty()
```


La siguiente consulta utiliza el método `aggregate` para mostrar los documentos de **Puesto** que se relacionan con la colecciones **Categoria y Departamento**, mediante el campo **Categoria._id y Departamento._id** respectivamente, esto por cuanto estos campos se habían incluido originalmente en una lista de `{}`.

```
db.Puesto.aggregate([
  { $lookup:
    {
      from: 'Categoria',
      localField: 'Categoria._id',
      foreignField: '_id',
      as: 'comments'
    }
  },
  {
    $unwind: '$comments'
  },
  {
    $lookup:{
      from: 'Departamento',
      localField: 'Departamento._id',
      foreignField: '_id',
      as: 'comments2'
    }
  }
]).pretty()
```

Creación y utilización de índices

La siguiente sentencia permite crear un índice para la colección Empleado y por el campo de nombre.

```
db.getCollection("Empleado").createIndex({"nombres.nombre":1});
```

La siguiente sentencia permite crear un índice para la colección Empleado y por los campos nombre, apellido1, apellido2.

```
db.getCollection("Empleado").createIndex({"nombres.nombre":1,"nombres.apellido1":1,"nombres.apellido2":1});
```

La búsqueda utilizando el índice permite mejorar mucho los tiempos, incluso se pueden lograr 0.001 S

```
db.getCollection("Empleado").find({"nombres.nombre":"Carlos","nombres.apellido1":"Rojas","nombres.apellido2":"Coto" });
```

Se puede utilizar un campo de una colección para que sea un índice cuyo valor sea único.

```
db.getCollection("Puesto").createIndex({"Categoria._id":1},{unique:true});
```

En este caso, en la colección puesto se crea un índice para el campo Categoria._id.

Para comprobar que no permite el ingreso de valores duplicados intentaremos realizar el siguiente insert:

```
db.Puesto.insertOne({_id:7, desc_puesto: "miscelaneo  
sucursal",salario:250000.00,Categoria:{_id:7},Departamento:{_id:1}});
```

La siguiente sentencia permite borrar un índice en particular:

```
db.getCollection("Puesto").dropIndex("Categoria._id_1");
```